

Reviewer Pack — Minimal Symplectic Volume to Communication Complexity Rank V...

Entry b064a4253200 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 14:09:39 UTC

Minimal Symplectic Volume to Communication Complexity Rank Variance Ratio

Entry ID: b064a4253200 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 14:09:39 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For all instances of k -CNF satisfiability problems, the ratio of the minimal symplectic volume of their associated projective space embeddings to the variance of their communication complexity rank is upper bounded by a constant, i.e., $|V(\text{min_sym_vol}) / \text{Var}(\text{Comm_Rank})| \leq c$.

Rationale (proposer's reasoning):

Symplectic geometry has been underutilized in complexity theory. By considering symplectic volumes, we may uncover structural properties related to the geometric nature of communication processes. This conjecture suggests that the complexity of communication might be inherently constrained by the geometric structure of problem representations.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d24b5e0132a2df5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed ratio of minimal symplectic volume to communication complexity rank variance for all k -CNF instances is less than or equal to a predefined constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "communication complexity" AND "matrix rank" - "projective space embedding" IN SYMPLECTIC Geometry AND communication complexity" - "minimal symplectic volume" AND "rank variance" AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1002.3099v3] Tamed Symplectic forms and SKT metrics - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def projective_space_embedding(cnf):
        n = max(abs(x) for x in cnf[0]) # Assuming each clause is a tuple of literals
        return n

    def communication_complexity_rank(cnf):
        m, n = len(cnf), len(cnf[0])
        matrix = [[1 if literal in clause else 0 for literal in range(1, n + 1)] for clause in cnf]

        # Gaussian elimination to find the rank
        rank = 0
        for i in range(n):
            pivot_row = next((j for j in range(rank, m) if matrix[j][i] != 0), None)
            if pivot_row is not None:
                matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
                rank += 1
                for j in range(m):
                    if j != rank - 1:
                        factor = matrix[j][i] / matrix[rank - 1][i]
                        for k in range(n):
                            matrix[j][k] -= factor * matrix[rank - 1][k]

        return rank
```

```

def variance(lst):
    mean = sum(lst) / len(lst)
    return sum((x - mean) ** 2 for x in lst) / len(lst)

def min_symplectic_volume(n):
    # Placeholder for the actual computation
    return n * (n + 1) // 2

cnf = []
n, m = random.randint(5, 30), random.randint(10, 30)
for _ in range(m):
    clause = tuple(random.sample(range(-n, -1) + list(range(1, n + 1)), random.randint(1, n)))
    cnf.append(clause)

sym_vol = min_symplectic_volume(n)
comm_rank = communication_complexity_rank(cnf)
var_comm_rank = variance([comm_rank] * 30) # Assuming all ranks are the same for simplicity

if var_comm_rank == 0:
    return {
        "metric_name": "Symplectic Volume to Communication Complexity Rank Variance Ratio",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Variance of communication complexity ranks is zero"
    }

ratio = abs(sym_vol / var_comm_rank)
return {
    "metric_name": "Symplectic Volume to Communication Complexity Rank Variance Ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio <= 10, # Placeholder constant c
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "Ratio exceeds constant bound"

```

```
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c07a95c0.py", line 88, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c07a95c0.py", line 55, in run_trial
    clause = tuple(random.sample(range(-n, -1) + list(range(1, n + 1)), random.randint(1, n)))
            ~~~~~^~~~~~
```

TypeError: unsupported operand type(s) for +: 'range' and 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a `TypeError`, which prevented it from producing any data to evaluate the conjecture. | next: Investigate and fix the error in the test code to ensure it can run successfully and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13278
2	preregistration	ollama_remote	glm4:latest	0	0	9000
3	novelty	ollama_remote	glm4:latest	0	0	8098
4	novelty	ollama_remote	glm4:latest	0	0	9319
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28158
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12554
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11831
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10064
9	judge	ollama_remote	glm4:latest	0	0	14770

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117072 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b064a4253200.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b064a4253200.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b064a4253200.tar.gz` (if generated)